

Entertainment Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Maitry Banduke	EN22CS301571
2	Mohd Quasim	EN22CS301603
3	Mrunali Kamerikar	EN22CS301618
4	Muskan Asija	EN22CS301623
5	Nikhil Kumar Singh	EN22CS301648

Group Name: GROUP 03 D6

Project Number: AAI-14

Industry Mentor Name:

University Mentor Name: Dr. Nishant Shrivastava

Academic Year: 2026

Table of contents

Sr. No.	Section / Chapter	Page No.
1	Problem Statement & Objectives	3-6
1.1	Problem Statement	3
1.2	Project Objectives	3-5
1.3	Scope of the Project	5-6
2	Proposed Solution	7-17
2.1	Key Features	7-8
2.2	Overall Architecture / Workflow	9-13
2.2.1	DFD (Data Flow Diagram)	9-10
2.2.2	Use Case Diagram	10-11
2.2.3	Class Diagram	11-12
2.2.4	Activity Diagram	12-13
2.3	Tools & Technologies Used	13-17
3	Results & Output	18-24
3.1	Outputs (Screenshots)	18-21
3.2	Reports (Dashboards / Models)	21-24
3.3	Key Outcomes	24
	Conclusion	25
	Future Scope & Enhancements	26

Chapter 1 Problem Statement & Objectives

1.1 Problem Statement

Modern entertainment platforms like Netflix, Amazon Prime, and IMDb use advanced Artificial Intelligence systems to provide highly personalized movie and show recommendations. These systems analyze user behavior such as watch history, ratings, search patterns, and preferences to suggest content that matches individual interests. However, most academic AI projects only build simple recommendation systems that suggest movies based on basic factors like genre similarity or high ratings. They do not include deeper personalization, intelligent reasoning, memory, adaptability, or explainability.

The main problem addressed in this project was to design and develop a production-level Agentic AI entertainment assistant that behaves like a smart digital companion rather than just a basic recommender. The system needed to understand user intent written in natural language, such as asking for a "light movie for the weekend" or "a movie similar to Inception but easier to understand." Instead of only matching keywords, the system had to understand the meaning behind the request and respond intelligently.

Another important challenge was personalization and learning. The system should not treat users as static profiles. It must learn from user ratings and interactions over time. If a user starts watching more thriller movies, the system should quickly adapt and recommend more thrillers. If the user dislikes a certain genre, the system should reduce similar suggestions. This continuous learning makes the system dynamic and user-focused.

The project also required explainable recommendations. Instead of just suggesting a movie, the system should clearly explain why it was recommended, such as "Because you rated similar action movies highly" or "This matches your interest in motivational dramas." Providing explanations builds user trust and transparency.

Additionally, the system needed to support multiple users, each with separate profiles, watch histories, and preferences. It also had to generate summaries, review highlights, and reflections, making it more than just a recommendation engine. Overall, the challenge was to create an intelligent, adaptive, and explainable AI system that combines data retrieval, reasoning, personalization, and generative capabilities to deliver a real-world entertainment planning experience.

1.2 Project Objectives

The main objective of this project is to develop a smart, real-world level Entertainment Planning Assistant that works like modern streaming platforms but is built using advanced AI concepts. Below is a detailed explanation of each objective.

- **To Build a Retrieval-Augmented Generation (RAG) Based Entertainment Assistant**

The first objective of this project is to build an entertainment assistant using the RAG (Retrieval-Augmented Generation) approach. In simple words, this means the system will first search for correct and relevant movie information from a database, and then generate a smart and meaningful response using AI. Instead of giving random or general answers, the assistant will combine real data with intelligent text generation. This helps in giving accurate recommendations, summaries, and explanations just like a real streaming platform assistant.

- **To Implement a Personalized Ranking Engine Using Semantic Similarity and User Preferences**

The second objective is to create a smart ranking system that suggests movies based on user interests. The system will understand the meaning of the user's query, not just the words. It will compare the user's request with movie descriptions using semantic similarity and check the user's past ratings and preferences. Then it will calculate a score for each movie and rank them accordingly. This ensures that the recommendations are personalized and more relevant to each individual user.

- **To Integrate a Multi-User Login System with Persistent Memory**

Another important objective is to allow multiple users to use the system. Each user will have their own account with login and registration features. The system will store their watch history, ratings, and preferences separately. Even after logging out, their data will be saved in the database. This is called persistent memory. It helps the system remember user choices and provide better recommendations in the future.

- **To Design a Frontend Interface Using Streamlit**

The project also aims to create a simple and user-friendly interface using Streamlit. The frontend will allow users to log in, enter their movie preferences, and view recommendations easily. The interface will clearly display movie titles, summaries, ratings, and explanation of scores. Streamlit makes it easier to design a clean and interactive application without complex frontend coding.

- **To Develop an Explainable AI Recommendation Engine with Scoring Breakdown**

This objective focuses on making the recommendation system transparent. Instead of just showing movie suggestions, the system will also explain why each movie was

recommended. It will show how the final score was calculated based on factors like genre match, similarity score, user preferences, and ratings. This explanation builds trust and helps users understand the reasoning behind the recommendations.

- **To Simulate a Real-World Netflix-Style Recommendation Workflow**

The final objective is to make the system work like a real streaming platform. The user will log in, search or ask for recommendations, receive personalized suggestions, and then give ratings or feedback. The system will learn from this feedback and improve future recommendations. This complete workflow makes the project practical and similar to real-world platforms, rather than just a simple academic model.

The overall objective of this project is to combine Retrieval-Augmented Generation, personalization, semantic search, multi-user support, explainable AI, and an interactive interface into one integrated system. By achieving these objectives, the project demonstrates how advanced AI techniques can be applied to build a smart, adaptive, and user-friendly entertainment recommendation platform similar to real-world streaming services.

1.3 Scope of the object

The scope of this project defines the features that are included in the system and also clearly mentions what is not included. The project focuses on developing an intelligent entertainment recommendation assistant that provides personalized movie suggestions based on user preferences and behavior. It includes a movie recommendation system that suggests movies using natural language queries, genre preferences, similarity matching, and ratings. The system uses a personalized ranking engine that calculates a score for each movie based on factors such as genre match, semantic similarity, user ratings, and overall popularity. Movies are then ranked according to this score to ensure that the most relevant results appear first.

Another important part of the project is genre learning and rating-based improvement. The system learns from user interactions and ratings over time. If a user frequently rates action movies highly, the system increases recommendations from that genre.

Similarly, if certain genres receive low ratings, the system reduces similar suggestions. This adaptive mechanism helps the assistant improve continuously and provide better personalized results. The project also includes explainable scoring, where the system clearly shows why a particular movie was recommended. It provides a breakdown of the scoring factors so users

understand the reasoning behind the suggestions. Additionally, the system supports multiple users with separate login accounts, stored preferences, and persistent memory, ensuring that each user receives independent and customized recommendations.

However, the project does not include real-time integration with external APIs such as live IMDb or streaming platform databases. All data used in the system is locally stored for simulation purposes. It also does not include cloud deployment, large-scale distributed architecture, or real-time production-level scaling. Features like high-performance server infrastructure, global streaming integration, and large-scale user traffic handling are outside the current scope.

These advanced features can be added in the future to expand the system into a fully commercial-grade platform.

Chapter 2 : Proposed Solution

The proposed solution is an Agentic AI-based entertainment assistant that combines retrieval, personalized ranking, memory, and language generation in one system. First, the user logs in through a Streamlit web interface and enters a query like "Recommend thriller movies" or "Summarize Interstellar." The system understands the user's intent and then retrieves relevant movies using semantic search powered by Chroma and Sentence Transformers, which matches meaning instead of just keywords. After that, a ranking engine calculates a personalized score based on similarity, ratings, genre preferences, and user history. Finally, a language model generates a clear response and displays it to the user, while updating the user's memory to improve future recommendations.

2.1 Key features

The system includes several important features that make it intelligent, personalized, and user-friendly. These features work together to provide accurate movie recommendations, understand user preferences, learn from ratings, and avoid repetition. It also ensures transparency by explaining recommendation scores and allows multiple users to have separate profiles. With semantic search, memory updates, summary generation, and an interactive web interface, the system functions like a smart entertainment assistant rather than a basic recommendation tool.

- **Multi-user Login System**

The system allows multiple users to register and log in with their own credentials. Each user has a separate account where their watch history, ratings, search queries, and genre preferences are stored independently. This ensures complete personalization, meaning the recommendations shown to one user are not influenced by another user's activity. It also makes the system more realistic and suitable for real-world usage where many users interact with the platform at the same time.

- **Personalized Ranking System**

Before recommending movies, the system calculates a personalized score for each available movie. This score is not random; it is based on several important factors such as the user's past ratings, preferred genres, similarity to the current query, and overall movie ratings. The movies with higher personalized scores appear at the top of the recommendation list. This ranking approach ensures that suggestions are more relevant, meaningful, and tailored to the user's interests.

- **Semantic Similarity Search**

Instead of relying only on exact keyword matching, the system uses semantic similarity search to understand the meaning behind the user's query. For example, if a

user asks for a “mind-bending sci-fi movie,” the system can recommend movies with similar themes even if those exact words are not used in the description. This improves the accuracy of recommendations and allows the system to handle natural language queries more intelligently.

- **Rating-Based Recommendation Improvement**

The system continuously improves by learning from user ratings. Whenever a user rates a movie, the system updates their preference profile. Movies similar to highly rated content are more likely to be recommended in the future, while those similar to low-rated movies are less likely to appear. This feedback mechanism helps the assistant become smarter and more aligned with the user’s taste over time.

- **Genre Preference Learning**

The system tracks the genres that a user watches and rates most frequently. If a user regularly watches and gives high ratings to action or thriller movies, the system gradually increases the priority of those genres in future recommendations. Over time, it builds a clear understanding of the user’s favorite categories and adjusts suggestions accordingly, making the recommendations more personalized and accurate.

- **History Penalty to Avoid Repetition**

To ensure variety and freshness in recommendations, the system applies a history penalty to movies that the user has already watched or rated. This reduces the chances of the same movies appearing repeatedly in the recommendation list. By lowering the score of previously seen content, the system encourages discovery of new and diverse movies while still considering user preferences.

- **Explainable Scoring Breakdown**

The system provides transparency by explaining how each recommendation score is calculated. It shows the contribution of factors such as semantic similarity, genre match, user preference weight, and overall ratings. Instead of acting like a “black box,” the system clearly communicates why a particular movie was suggested. This improves user trust and helps users understand the logic behind recommendations.

- **Summary and Review Generation**

In addition to recommending movies, the assistant can generate short summaries and highlight important review points. This helps users quickly understand the storyline, main themes, and general public opinion about a movie. By providing concise and informative descriptions, the system supports better decision-making without requiring users to search elsewhere for information.

2.2 Overall Architecture / Workflow

The overall architecture of the system is designed in a structured and modular way to ensure smooth functioning and intelligent decision-making. Each component in the system has a specific role, such as handling user interaction, understanding queries, retrieving relevant data, ranking results, generating responses, and storing memory. All these components work together in a step-by-step workflow to provide personalized, explainable, and adaptive movie recommendations. This organized architecture makes the system efficient, scalable, and similar to real-world entertainment platforms.

2.2.1 DFD (Data Flow Diagram)

A **Data Flow Diagram (DFD)** is a graphical representation used in system analysis and design to illustrate how data moves through a system. It focuses specifically on the flow of information rather than the physical implementation, technical architecture, or control logic. In the context of the **Entertainment Planning Assistant Agent**, the DFD explains how a user's entertainment-related query is received, processed, stored, retrieved, and transformed into a meaningful response.

DFD Level-0

The Level-0 Data Flow Diagram presents the system as a single high-level process and focuses on external interactions. It shows how the user provides inputs such as login credentials, entertainment queries, and ratings to the Entertainment Planning Assistant, which processes the data and returns outputs like recommended movies, summaries, reviews, and explanations. The Admin supplies movie data and configuration updates, emphasizing the overall input-output flow without internal details.



Figure 1 : DFD Level 0

DFD Level-1

DFD Level 1 decomposes the system into detailed internal processes to show how data flows between them. It includes steps such as intent classification, embedding generation, similarity search in the vector database, user context retrieval from memory, ranking based on preferences and history, and response generation using an LLM. This diagram clearly demonstrates how different AI components collaborate to produce accurate and personalized results.

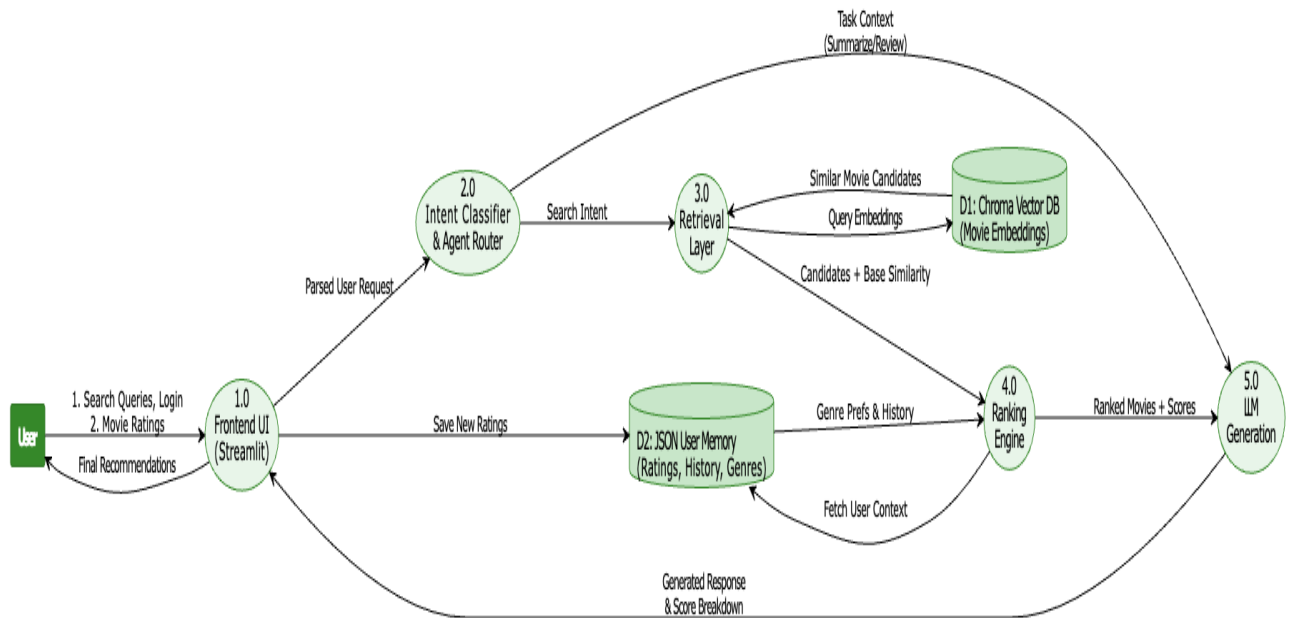
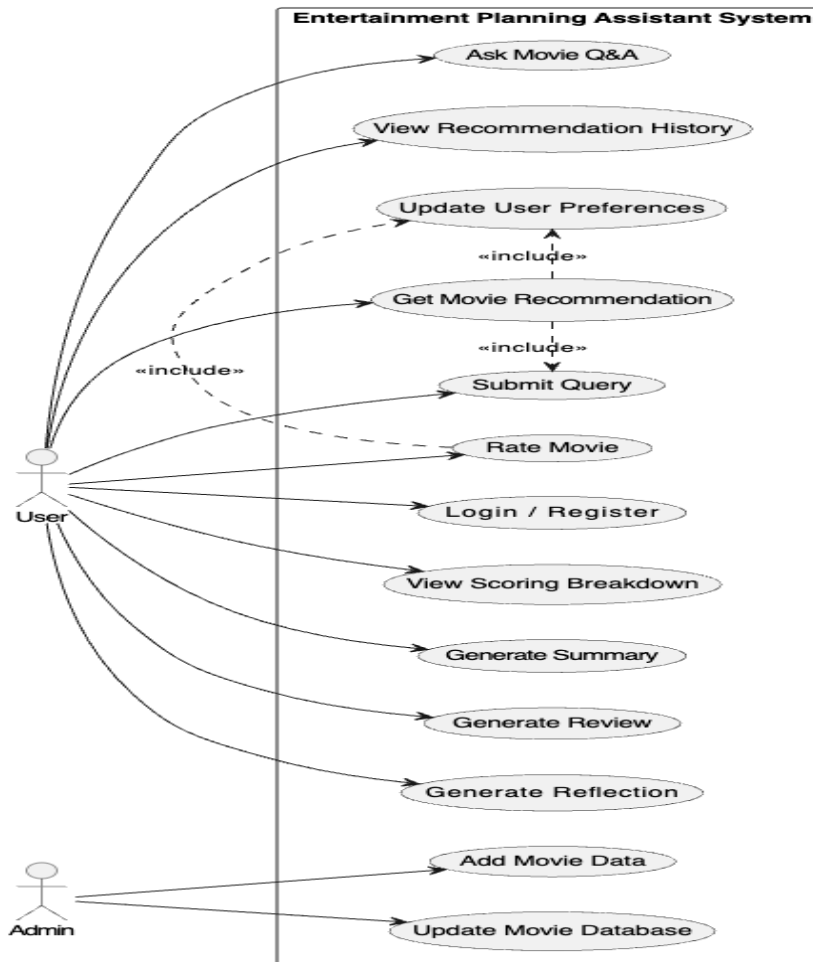


Figure 2 : DFD Level 1

2.2.2 Use Case Diagram

The use case diagram describes the functional behavior of the system from the perspective of external actors. The **User** can log in or register, submit queries, request movie recommendations, generate summaries, reviews, and reflections, rate movies, and view recommendation history and scoring explanations. The **Admin** interacts with the system to add new movie data, update the movie database, and maintain system accuracy and performance.



2.2.3 Class Diagram

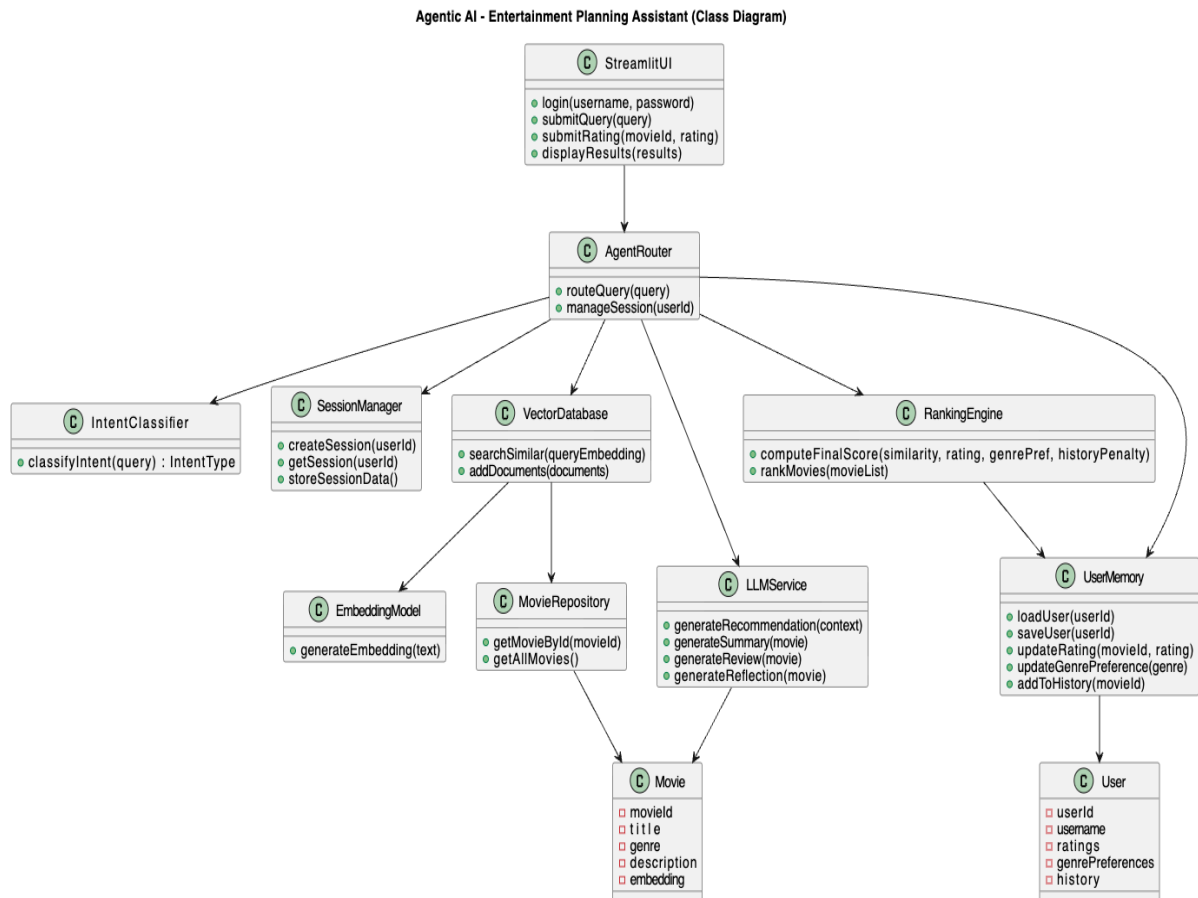


Figure 4 : Class Diagram

2.2.4 Activity Diagram

The activity diagram illustrates the dynamic workflow of the system in a step-by-step manner. It begins with the user opening the application and logging in, followed by query submission and intent detection. Based on the detected intent, the system either generates recommendations or produces summaries, reviews, or Q&A responses, and finally updates user ratings, genre preferences, and interaction history before saving the updated user memory.

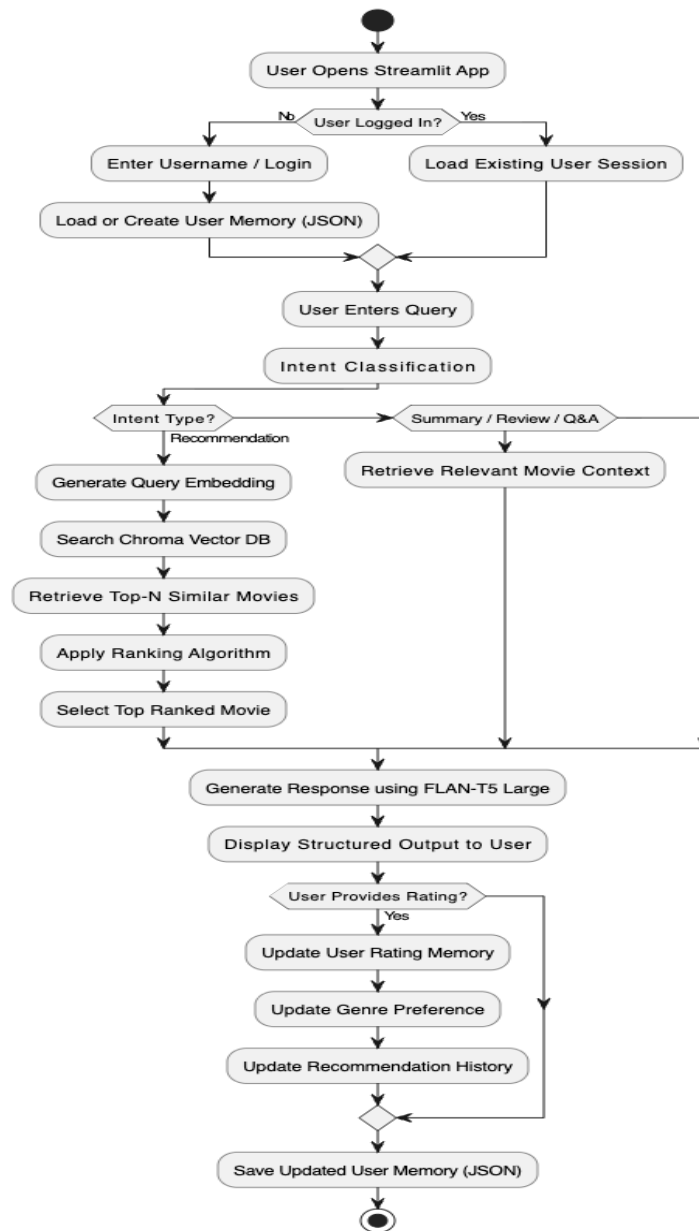


Figure 5 : Activity Diagram

2.3 Tools & Technologies Used

- Programming Language: Python**

Python is the core programming language used to build the entire system. It is widely preferred in Artificial Intelligence, Machine Learning, and data-driven applications because of its simplicity, readability, and powerful ecosystem. Python allows easy integration of AI models, databases, APIs, and web interfaces within a single environment.

In this project, Python is used for:

- Implementing the complete backend logic.
- Writing the ranking algorithm.
- Managing user authentication and sessions.
- Connecting the frontend with the backend.
- Handling vector database operations.

Python libraries such as NumPy, JSON handling modules, and ML-based packages help in performing mathematical calculations, similarity scoring, and data manipulation efficiently. Its flexibility allows smooth coordination between retrieval, ranking, memory, and generation layers.

- **Framework: LangChain**

LangChain is used as the orchestration framework to manage communication between different AI components. It is specifically designed to build applications that use Large Language Models (LLMs) combined with external data sources.

In this project, LangChain performs the following functions:

- Manages prompt templates for structured response generation.
- Connects the language model with the vector database.
- Handles retrieval-augmented generation (RAG) workflows.
- Organizes multi-step operations (retrieve → rank → generate).
- Controls memory integration with user sessions.

LangChain helps in chaining multiple operations together so that the system works in a structured pipeline. Without LangChain, managing interactions between retrieval, ranking, and generation would be more complex.

- **Frontend: Streamlit**

Streamlit is used to create the web-based user interface. It allows developers to build interactive web applications using only Python, without requiring HTML, CSS, or JavaScript.

In this project, Streamlit is responsible for:

- Creating login and registration forms.
- Managing user authentication interface.
- Accepting natural language queries.
- Providing rating input options.
- Displaying recommended movies in an organized layout.

Streamlit enables real-time interaction between the user and the backend. It automatically refreshes and updates outputs when inputs change, making the system responsive and interactive. This makes the project look like a professional web application.

• **Vector Database: Chroma**

Chroma is used as the vector database to store and retrieve embeddings efficiently. Unlike traditional databases that store text and numbers, vector databases store high-dimensional numerical representations of text.

In this project, Chroma is used for:

- Storing embeddings of movie descriptions.
- Performing fast similarity search.
- Retrieving top-k similar movies based on query embedding.
- Returning similarity distance scores.
- When a user enters a query:
- The query is converted into an embedding.
- Chroma compares it with stored movie embeddings.
- It calculates similarity distance.
- It returns the most relevant movies.

Chroma ensures high-speed retrieval and efficient semantic matching. It makes the recommendation system intelligent by enabling meaning-based search rather than simple keyword matching.

• **Embeddings: Sentence Transformers**

Sentence Transformers are used to convert text into dense numerical vectors called embeddings. These embeddings capture the contextual and semantic meaning of text.

In this project:

- Every movie description is converted into an embedding.
- User queries are also converted into embeddings.
- Cosine similarity or distance metrics are used to compare embeddings.

Example:

If a user asks for “emotional drama about family,” the embedding captures the meaning of the entire sentence. Even if a movie description uses different words like “heart-touching story of relationships,” the system can still match them based on semantic similarity.

- Sentence Transformers improve:
- Context understanding.
- Natural language handling.
- Recommendation accuracy.
- Meaning-based retrieval.

- **Language Model: FLAN-T5 Large**

FLAN-T5 Large is the Large Language Model (LLM) used for generating natural language responses. It is an instruction-tuned model capable of understanding prompts and generating structured outputs.

In this project, FLAN-T5 Large is used to:

- Generate movie summaries.
- Create recommendation explanations.
- Provide score breakdown descriptions.
- Compare movies.
- Highlight review points.
- Format final responses clearly.

The system provides structured prompts to guide the model. For example:

- “Explain why this movie is recommended based on user preference.”
- “Summarize the movie in 4 lines.”
- “Compare Movie A and Movie B.”

The model generates human-like, clear, and coherent text. This makes the system interactive and conversational rather than just data-driven.

- **Storage: JSON-Based Persistent Storage**

JSON-based storage is used to store user-specific data in a lightweight and structured format. JSON files are easy to read, write, and update using Python.

In this project, JSON storage keeps:

- User login credentials.
- Movie ratings given by users.
- Genre preference counts.
- Recommendation history.
- Previously watched movies.

Functions of JSON storage:

- Updates data whenever user rates a movie.
- Stores user activity persistently.
- Loads user profile during login.
- Applies history penalty during ranking.
- Tracks genre learning over time.

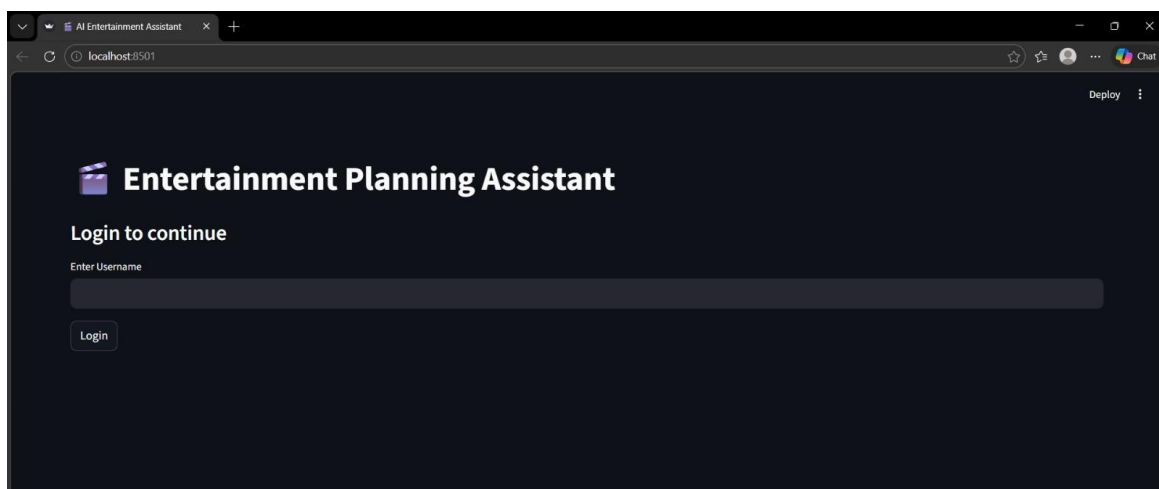
Persistent storage ensures that data remains saved even after closing the application. This enables long-term personalization and continuous learning without requiring complex databases.

Chapter 3: Results & Output

3.1 Outputs

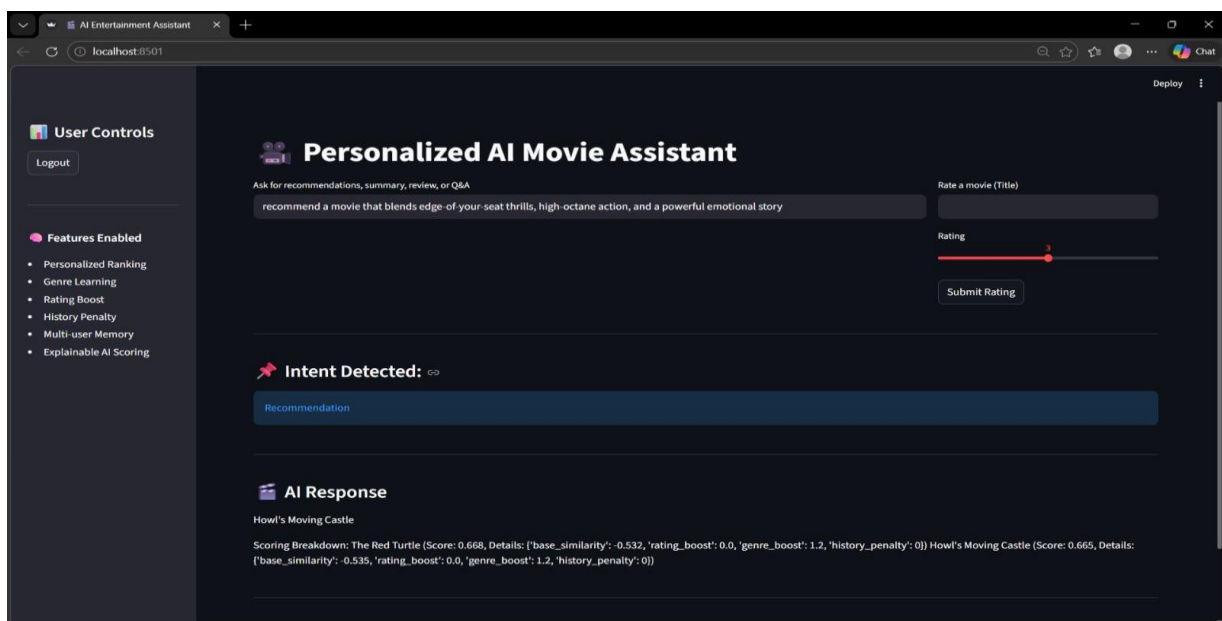
3.1.1 Login

This allows users to securely access the system using their credentials. It enables personalized features by identifying the user and loading their preferences and interaction history.



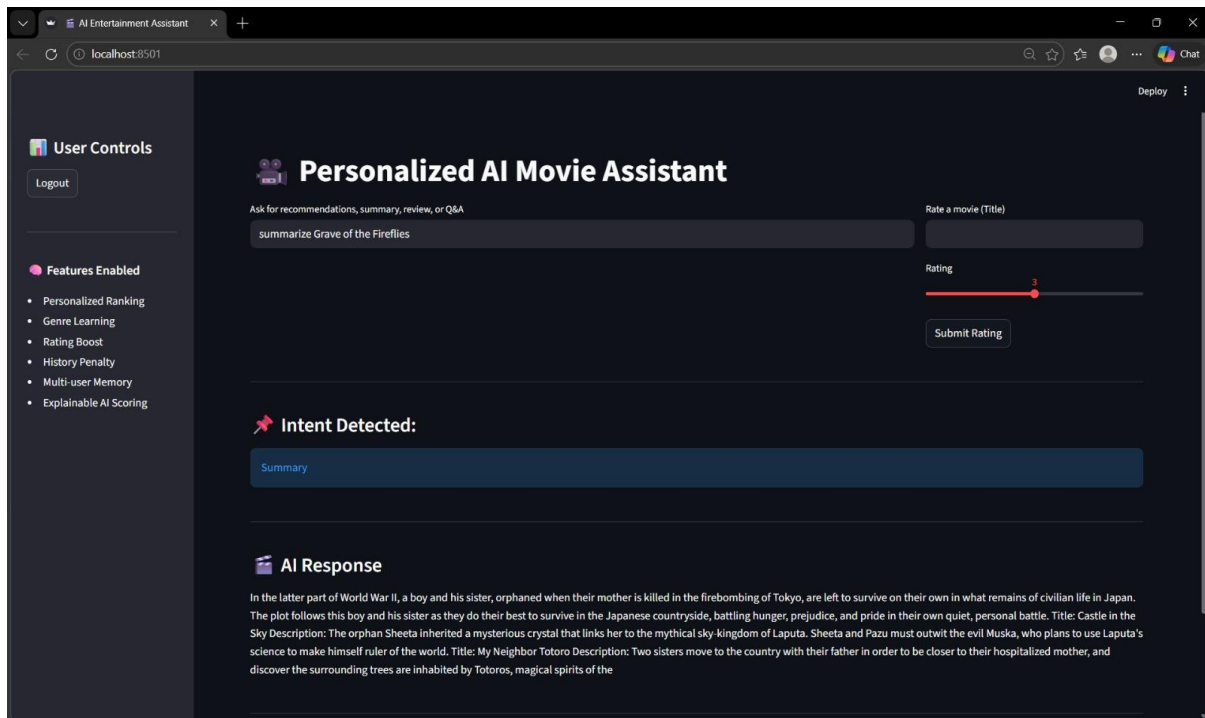
3.1.2 Recommendation result with scoring breakdown

It provides movie suggestions based on user preferences and detected intent. The system explains each recommendation by showing score components such as similarity, genre boost, rating influence, and history penalty for transparency.



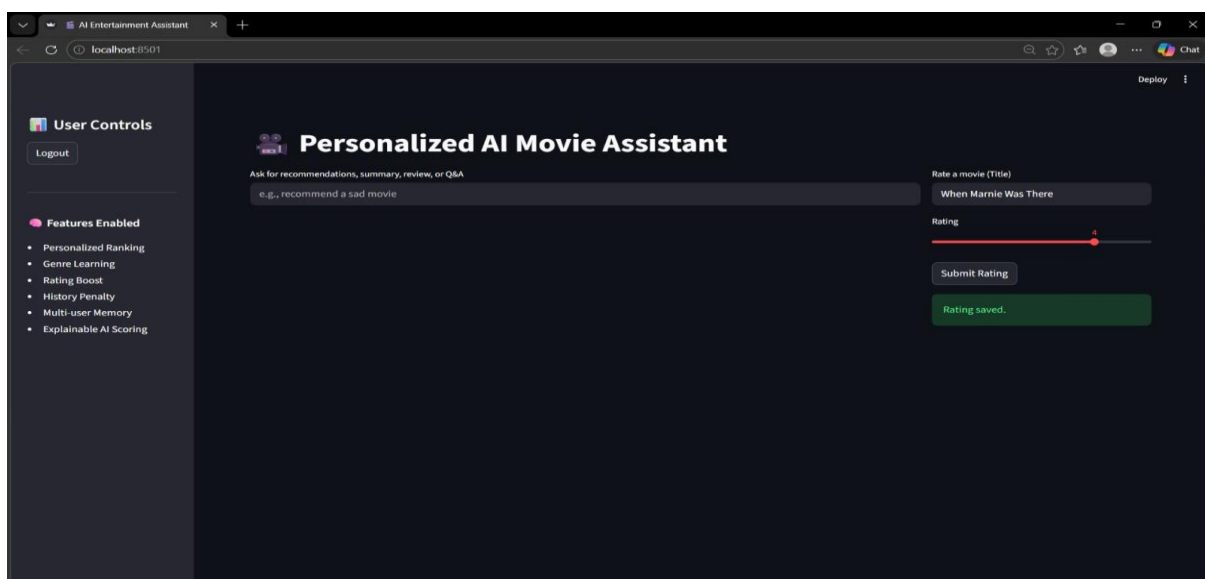
3.1.3 Summary generation

This allows the system to detect a user's request for a movie summary and retrieve relevant movie information. The AI then uses an LLM to generate a concise, clear summary of the movie's storyline for quick understanding.



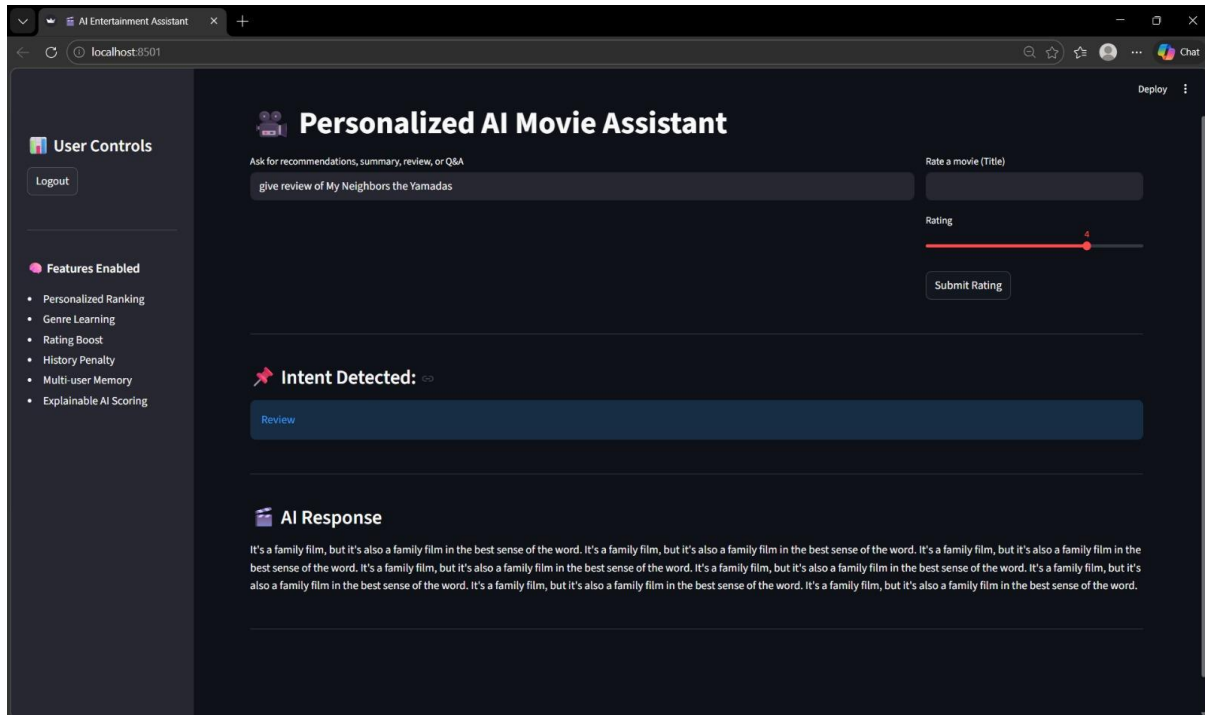
3.1.4 Rating submission

The below image shows that rating submission feature, where the user selects a movie and provides a rating using a slider. The submitted rating is saved and used to improve personalized recommendations and learning of user preferences.



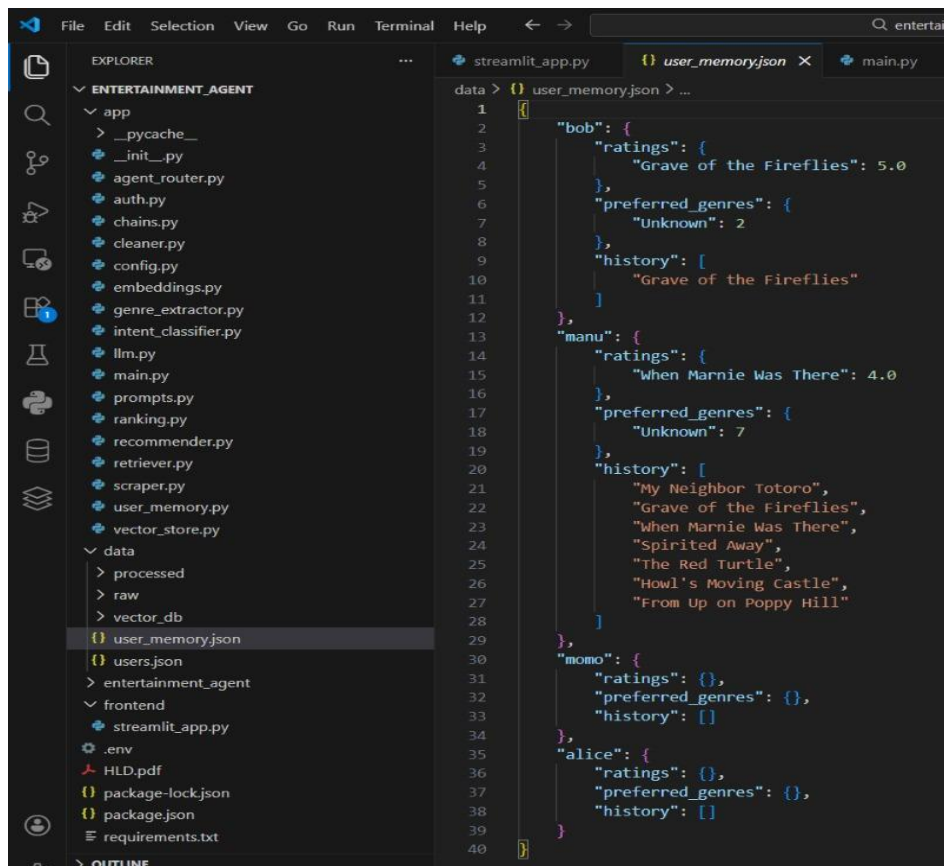
3.1.5 Review of the movie

This screen shows the Personalized AI Movie Assistant generating an AI-based movie review after detecting the user's intent. It demonstrates intent recognition, personalized features, and automated review generation in the system.



3.1.6 Multi-user behavior difference, User memory JSON structure

The below image shows the **multi-user memory system**, where each user has a separate JSON record storing ratings, preferred genres, and viewing history. It enables the system to learn **different behaviors per user** and generate personalized recommendations accordingly.



The screenshot shows a VS Code editor with a project named 'ENTERTAINMENT_AGENT'. The Explorer sidebar on the left shows the project structure, including folders like 'app', 'data', and 'frontend', and files like 'user_memory.json'. The main editor area displays the content of 'user_memory.json', which is a JSON array containing user profiles for 'bob', 'manu', 'momo', and 'alice'. Each profile includes 'ratings', 'preferred_genres', and 'history'.

```
data > {} user_memory.json > ...
1  [
2    {
3      "bob": {
4        "ratings": {
5          "Grave of the Fireflies": 5.0
6        },
7        "preferred_genres": {
8          "Unknown": 2
9        },
10       "history": [
11         "Grave of the Fireflies"
12       ]
13     },
14     "manu": {
15       "ratings": {
16         "When Marnie Was There": 4.0
17       },
18       "preferred_genres": {
19         "Unknown": 7
20       },
21       "history": [
22         "My Neighbor Totoro",
23         "Grave of the Fireflies",
24         "When Marnie Was There",
25         "Spirited Away",
26         "The Red Turtle",
27         "Howl's Moving Castle",
28         "From Up on Poppy Hill"
29       ]
30     },
31     "momo": {
32       "ratings": {},
33       "preferred_genres": {},
34       "history": []
35     },
36     "alice": {
37       "ratings": {},
38       "preferred_genres": {},
39       "history": []
40     }
41   ]
}
```

3.2 Reports / dashboards / models

3.2.1 Models

- **Embedding Model**

The system uses a Sentence Transformer-based embedding model to convert textual data into numerical vector representations called embeddings. This model transforms both movie descriptions and user queries into high-dimensional vectors that capture contextual and semantic meaning. Instead of relying on keyword matching, the embedding model understands the intent and overall meaning of a sentence. These embeddings are then used for similarity comparison, enabling the system to recommend movies based on conceptual relevance. This model is based on deep learning and the Transformer architecture, making it highly effective for natural language understanding tasks.

- **Large Language Model – FLAN-T5 Large**

The system uses FLAN-T5 Large as its primary Large Language Model (LLM) for generating natural language responses. This is an instruction-tuned, transformer-based sequence-to-sequence model capable of understanding structured prompts and generating coherent outputs. It is responsible for producing movie summaries,

recommendation explanations, comparisons between movies, and formatted responses. The LLM enhances the system by converting raw recommendation data into human-readable explanations, making the application conversational and user-friendly. Its instruction-following capability ensures clear and structured outputs.

- **Retrieval Model**

The system employs a semantic retrieval model based on vector similarity search. After generating embeddings for the user query and stored movie descriptions, the system calculates similarity scores—typically using cosine similarity—to identify the most relevant movies. This retrieval model ranks movies based on how closely their embeddings match the query embedding. It enables fast and accurate semantic search, ensuring that recommendations are context-aware rather than based on exact word matches. This approach significantly improves recommendation relevance and efficiency.

- **Rule-Based Ranking Model**

In addition to AI models, the system includes a custom rule-based ranking model that calculates the final recommendation score. This scoring algorithm combines multiple factors such as semantic similarity score, genre preference boost, rating weight, and history penalty. By integrating these elements, the model ensures personalized and balanced recommendations. Unlike purely data-driven models, this rule-based approach makes the system transparent and explainable, as users can understand how the final score is computed. It plays a key role in enhancing personalization and trust in the recommendation process.

3.2.2 Reports

- **Personalized Recommendation Report**

This report displays the final recommended movies for the user along with detailed scoring information. It includes the movie title, similarity score, genre match score, final ranking score, and explanation generated by the language model. The purpose of this report is to provide transparent and personalized results based on user preferences and query context. It helps users understand why a particular movie was recommended.

- **User Profile & Memory Report**

This report shows stored user data from the JSON memory structure. It includes user ratings, preferred genres, watched movies, and recommendation history. The purpose of this report is to demonstrate how the system learns from user behavior over time. It helps in tracking personalization and validating adaptive learning functionality.

- **Scoring Breakdown Report**

This report explains how the final recommendation score is calculated. It presents a breakdown of different components such as semantic similarity score, genre preference boost, rating weight, and history penalty. The objective of this report is to make the system explainable and transparent. It is especially important for academic evaluation as it shows the working logic of the ranking algorithm.

- **Semantic Similarity Ranking Report**

This report shows how movies are ranked based on vector similarity scores. It displays similarity distances between the user query embedding and stored movie embeddings. The purpose is to illustrate how semantic search works in the system. It highlights the effectiveness of embedding-based retrieval instead of traditional keyword search.

- **Activity / Interaction Report**

This report tracks user interactions such as login, search queries, ratings given, and recommendations viewed. It helps in analyzing system usage patterns and user engagement. This can be useful for performance monitoring and future improvement

- **System Workflow / Architecture Report**

This report includes diagrams such as:

- DFD (Level 0 & Level 1)
- Use Case Diagram
- Class Diagram
- Activity Diagram

It provides a structural and behavioral overview of the system. This report is mainly used for documentation and academic presentation.

3.2.3 Dashboards

- **User Dashboard**

This is the main interface where users search for movies and receive personalized recommendations. It displays recommended movies, similarity scores, final ranking score, summaries, and explanations. Users can also rate movies and view their recommendation history.

- **Recommendation Analytics Dashboard**

This dashboard shows the scoring breakdown of each recommended movie. It includes semantic similarity score, genre preference boost, rating weight, and history penalty. It helps in making the system transparent and explainable.

- **User Profile Dashboard**
This dashboard displays stored user information such as preferred genres, rated movies, and watch history. It demonstrates how the system learns from user behavior and provides personalized results.
- **System Workflow Dashboard**
This section includes diagrams like DFD (Level 0 & Level 1), Use Case Diagram, Class Diagram, and Activity Diagram. It explains the internal architecture and working process of the system

3.3 Key Outcomes

- **Retrieval-Augmented Generation (RAG)**
The system implements Retrieval-Augmented Generation (RAG), which combines semantic retrieval with language generation. Instead of relying only on the language model's internal knowledge, the system first retrieves relevant movie data from a vector database and then uses the language model to generate summaries and explanations. This improves accuracy, contextual relevance, and reliability of responses.
- **Explainable AI Scoring**
The recommendation process follows Explainable AI principles by showing how the final score is calculated. The scoring includes semantic similarity, genre preference boost, rating weight, and history penalty. By displaying this breakdown, the system ensures transparency and builds user trust while allowing clear academic evaluation of the ranking logic.
- **Multi-User Architecture**
The system supports multiple users with independent data storage. Each user has separate memory containing ratings, preferences, and history. When a user logs in, the system loads their specific data and updates it after interactions. This demonstrates scalable and real-world system design.
- **Full stack AI Application Development**
The project demonstrates full stack AI development by integrating frontend (Streamlit), backend (Python), AI models (embedding model and LLM), vector database, and persistent storage into one complete system. It showcases end-to-end AI application design from user interaction to intelligent output generation.

Conclusion

The Agentic AI – Entertainment Planning Assistant is designed to demonstrate how modern AI techniques can be combined to build an intelligent and practical recommendation system. Unlike traditional systems that rely only on keyword matching or simple filtering, this project uses semantic search, ranking algorithms, memory management, and language generation to create a more advanced solution.

The system follows an Agentic AI approach, meaning it performs multiple structured steps before generating a response. When a user enters a query, the system converts it into embeddings, retrieves relevant movies using a vector database, ranks them using personalization logic, and then generates explanations using a language model. This step-by-step reasoning reflects real-world AI architecture rather than a basic response system.

The project also implements Retrieval-Augmented Generation (RAG), where retrieved movie data is passed to the language model to generate accurate and contextual summaries. This improves reliability and prevents the model from generating unsupported information.

Personalization is another key strength of the system. It tracks user ratings, genre preferences, and history to tailor recommendations individually. Additionally, the scoring breakdown is displayed clearly, making the system explainable and transparent instead of a black-box model.

From a learning perspective, this project provided practical understanding of vector databases, embedding models, similarity ranking, recommendation algorithms, and full stack AI integration. It demonstrates modular backend design and multi-user architecture, making it scalable and production-oriented.

Overall, this system bridges the gap between academic-level implementation and real-world AI application development by combining intelligent retrieval, personalization, explainable scoring, and conversational AI into one complete solution.

Future Scope & Enhancements

- **Collaborative Filtering**

Recommends movies based on similarities in user ratings and behavior. It improves personalization by learning from community preferences instead of only individual choices.

- **Poster Image Integration**

Displays official movie posters along with titles and ratings, making the interface more attractive, engaging, and similar to real streaming platforms.

- **Real Movie API Integration (TMDB)**

Fetches real-time movie data such as ratings, genres, summaries, and posters. This ensures updated and realistic recommendations instead of static datasets.

- **FastAPI Production Backend (FastAPI)**

Provides a fast, scalable, and production-ready backend API with automatic documentation and efficient request handling.

- **Docker Deployment (Docker)**

Packages the application with all dependencies into containers, ensuring smooth deployment, portability, and environment consistency.